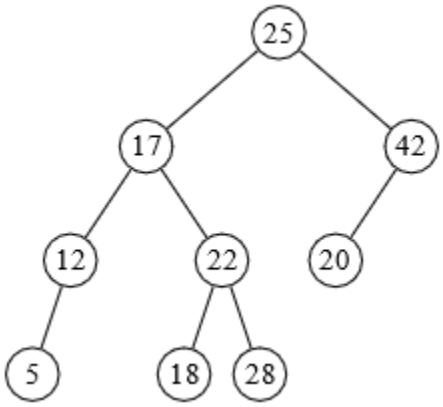
Xem demo cây nhị phân tìm kiếm và cây cân bằng

Binary Search Tree, AVL Tree - VisuAlgo

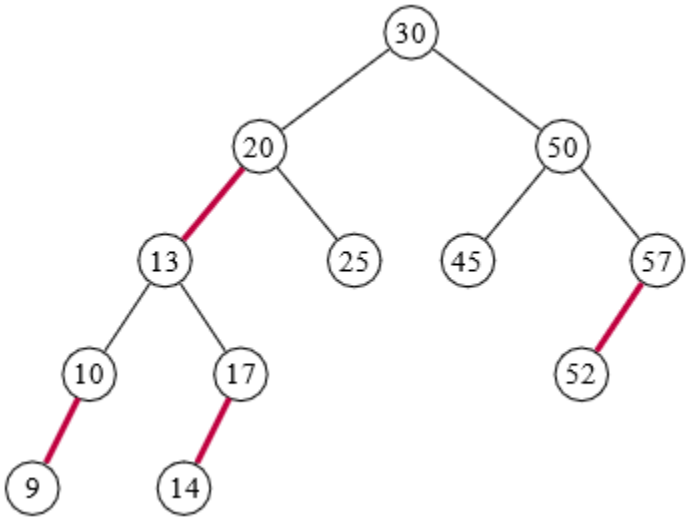
1. **COS226, Midterm f24, 4c**

Đây có phải một cây nhị phân tìm kiếm hợp lệ?



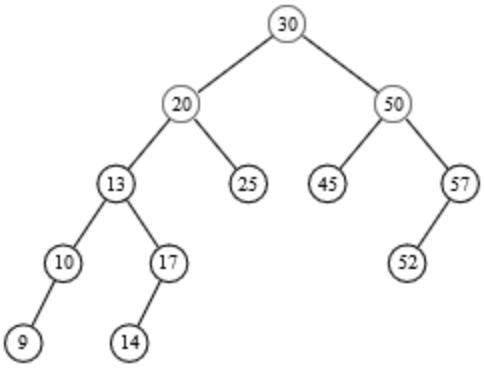
2. **COS226, Midterm f24, 5**

Cho cây đỏ đen nghiêng trái dưới đây

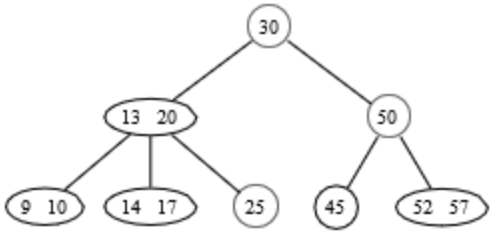


Cây trên ứng với cây 2-3 nào dưới đây:

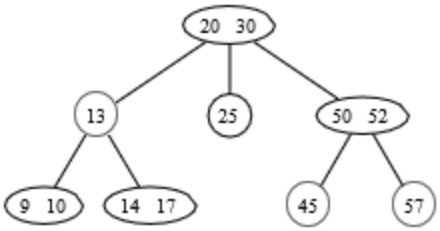
A.


30
20
50
13
25
45
57
10
17
52
9
14

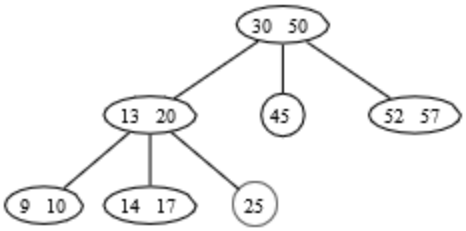
B.


30
13 20
50
9 10
14 17
25
45
52 57

C.


20 30
13
25
50 52
9 10
14 17
45
57

D.


30 50
13 20
45
52 57
9 10
14 17
25

Bài 1

Lấy mã nguồn chương trình và dữ liệu về các cài đặt (implementation) Binary Search Trees (BST.java) tại link mã nguồn môn học. Chạy lại chương trình với dữ liệu test.
Chú ý: Thư viện algs4.jar

Bài 3 Binary Search Tree : Insertion

Tóm tắt: Viết hàm Thêm/Chèn một đỉnh với giá trị cho trước vào cây tìm kiếm nhị phân.
Gợi ý:
Cách 1: Sử dụng đệ qui;
Cách 2: Dùng vòng lặp tìm vị trí cần chèn vào; rồi thêm vào.

Bài 4 Binary Search Tree : Lowest Common Ancestor

Tóm tắt: Tìm đỉnh tổ tiên chung gần nhất của 2 đỉnh cho trước. (Hai đỉnh con chắc chắn có trong cây tìm kiếm nhị phân).
Gợi ý: Sử dụng đệ qui.

Bài 5 Is This a Binary Search Tree?

Tóm tắt: Kiểm tra xem cây hiện tại có phải là cây tìm kiếm nhị phân hay không ?

Chú ý: Do lỗi của HackerRank, khi làm bài này phải chọn **Java 7**.

Gợi ý:
Cách 1: Duyệt cây theo thứ tự trong (in-order). Nếu dãy số (các giá trị khoá) thu được tăng dần thì đúng; nếu không là sai.
Cách 2: Sử dụng đệ qui. Cây tìm kiếm nhị phân khi:
a) Cây con trái và cây con phải cũng cùng là cây tìm kiếm nhị phân; và
b) Đỉnh cha có khoá lớn hơn khoá lớn nhất cây con bên trái và nhỏ hơn khoá nhỏ nhất cây con bên phải.

Bài 6 Self Balancing Tree

Tóm tắt: Viết hàm Chèn/Thêm một đỉnh với khoá cho trước vào cây tìm kiếm nhị phân cân bằng AVL. (Cây kết quả vẫn là cân bằng.)

(*) *Là các câu không cơ bản, bạn có thể bỏ qua mà không sợ trượt*

Goodrich & Tamassia, *Data structures & Algorithms in C++*, 2nd Ed. 2011

1. Nếu ta thêm vào một cây nhị phân tìm kiếm (BST) rỗng các key A, B, C, D, E theo thứ tự đó, ta sẽ thu được cây kết quả trông như thế nào?
2. Hãy thêm vào một cây BST rỗng các key sau theo đúng thứ tự đã cho: 30, 40, 24, 58, 48, 26, 11, 13. Hãy vẽ cây sau khi thêm từng key.
3. Hãy vẽ tất cả các cây BST chứa đúng 03 key 1, 2, 3.
4. Jack khẳng định rằng với một tập các key cho trước, thứ tự thêm các key đó vào cây không quan trọng, thứ tự nào cũng cho cây kết quả như nhau. Bạn hãy cho ví dụ để chứng minh rằng Jack đã sai.
5. Rose khẳng định rằng với một tập các key cho trước, thứ tự thêm các key đó vào cây đỏ đen (Red-Black tree) không quan trọng, thứ tự nào cũng cho cây kết quả như nhau. Bạn hãy cho ví dụ để chứng minh rằng Rose đã sai.
6. James khẳng định rằng với một tập các key cho trước, thứ tự thêm các key đó vào cây 2-3 không quan trọng, thứ tự nào cũng cho cây kết quả như nhau. Bạn hãy cho ví dụ để chứng minh rằng James đã sai.
7. Cho bộ key {1,2,3, 4,5 ,6,7,8,9,10,11,12,13,14,15}
    a. Hãy vẽ một cây 2-3 lưu trữ bộ key trên mà cần ít nút nhất
    b. Hãy vẽ một cây 2-3 lưu trữ bộ key trên mà cần nhiều nút nhất
8. Cho chuỗi key { 5, 16, 22, 45, 2, 10, 18, 30, 50, 12,1}, Hãy vẽ kết quả của việc thêm các key đó theo thứ tự đã cho vào
    a. Một cây 2-3 rỗng
    b. Một cây đỏ đen rỗng
9. Cho các tuyên bố sau về cây đỏ đen, (*)hãy chứng minh các câu đúng và tìm phản ví dụ cho mỗi câu sai
    a. Cây con của cây đỏ đen cũng là cây đỏ đen
    b. Nút anh em của một nút lá (nút lá là nút không có nút con) bao giờ cũng là nút lá hoặc nút đỏ
    c. Có duy nhất một cây 2-3 tương tương với một cây đỏ đen

d. Có duy nhất một cây đỏ đen tương đương với một cây 2-3

10. Xét một cây T chứa 100.000 nút. Hãy cho biết chiều cao trong trường hợp xấu nhất của T nếu như T là

    a. Một cây 2-3

    b. Một cây đỏ đen

    c. Một cây tìm kiếm nhị phân thông thường

11. Hãy nêu cách sử dụng một cây đỏ đen để sắp xếp n key theo thứ tự tăng dần với độ phức tạp n log n ở trường hợp tồi nhất

12. Hãy giải thích tại sao bạn sẽ nhận được kết quả giống hệt nhau nếu duyệt một cây tìm kiếm theo thứ tự inorder, bất kể cây đó được giữ ở dạng nào, cây BST thường, cây đỏ đen, hay cây 2-3.

Sedgewick & Wayne, *Algorithms*, 4th ed.

3.2.1. Hãy vẽ cây BST thu được khi bạn thêm các key `E A S Y Q U E S T I O N`, theo thứ tự đó vào một cây ban đầu rỗng. Cần bao nhiêu phép so sánh để xây dựng cây đó?

3.2.2 Hãy thêm các key sau theo thứ tự `A X C S E R H` vào một cây BST rỗng, vẽ cây kết quả.
Với những thứ tự nào thì sẽ thu được cây dạng tồi nhất (worst-case tree, khi nút nào cũng có 01 link null ngoại trừ nút dưới đáy có 2 link null) ? Hãy tìm 06 thứ tự key cho ra worst-case tree.

3.2.3 Hãy tìm 05 thứ tự của các key `A X C S E R H` mà khi thêm vào 1 cây BST rỗng sẽ tạo ra cây loại tốt nhất (best-case tree).

3.2.4 Giả sử có một cây BST có các key là các số nguyên từ `1` đến `10`, và ta cần tìm key `5`. Trong các chuỗi dưới đây, những chuỗi nào không thể là chuỗi các key được duyệt đến?

a. `10, 9, 8, 7, 6, 5`

b. `4, 10, 8, 7, 53`

c. `1, 10, 2, 9, 3, 8, 4, 7, 6, 5`

d. `2, 7, 3, 8, 4, 5`

e. `1, 2, 10, 4, 8, 5`

3.2.5 (*) Giả sử ta đã có một ước tính trước về tần suất truy nhập các key trong một cây BST, và ta có thể thêm key vào cây theo thứ tự tùy thích. Nên thêm key vào cây theo thứ tự nào thì tốt nhất? Thứ tự tăng dần, giảm dần, hay key nào hay được tìm kiếm thì thêm vào trước, hay là thứ tự nào khác? Hãy giải thích lời giải của bạn.

3.2.6 Với cài đặt cây BST đã cho, hãy bổ sung hàm `height()` tính độ cao của cây. Hãy cài theo 02 cách: một hàm đệ quy (sẽ cần thời gian tuyến tính và không gian bộ nhớ tỷ lệ thuận với độ cao), và một hàm

tương tự `size()` mà cần bổ sung 01 trường (biến) vào cho mỗi nút trên cây (sẽ cần thời gian hằng số và không gian bộ nhớ tuyến tính cho mỗi lần gọi).

3.2.7 Thêm vào cài đặt cây BST một hàm đệ quy `avgCompares()` với nhiệm vụ tính số phép so sánh trung bình mà một lần tìm kiếm ngẫu nhiên thành công cần đến (tổng độ sâu của mỗi nút trên cây chia cho số nút trên cây rồi cộng thêm 1). Hãy phát triển 2 cài đặt: một hàm đệ quy (thời gian tuyến tính và bộ nhớ tỉ lệ thuận với chiều cao cây), và một hàm tương tự `size()` cần thêm một trường (biến) vào mỗi nút trên cây (thời gian hằng số và không gian tuyến tính cho mỗi lần gọi).

3.2.8 Viết một hàm static `optCompares()` lấy một tham số `N` kiểu int và tính số phép so sánh cần đến cho một lần tìm kiếm thành công ngẫu nhiên trong một cây BST cân bằng hoàn hảo, là cây mà tất cả các link null đều nằm trên cùng một tầng nếu số link null là lũy thừa của 2, hoặc nằm trên một trong hai tầng đáy trong trường hợp còn lại.

3.2.9 Hãy vẽ tất cả các dạng cây BST khác nhau có thể thu được sau khi thêm `N` key vào một cây rỗng, với `N` = 2, 3, 4, 5, và 6.

3.2.10 Hãy viết chương trình `TestBST.java` để test/chạy thử các hàm `min()`, `max()`, `floor()`, `ceiling()`, `select()`, `rank()`, `delete()`, `deleteMin()`, `deleteMax()`, và `keys()` đã cho trong bài giảng.

3.2.11 (*)Có bao nhiêu dạng cây BST chứa N nút với chiều cao N? Có bao nhiêu cách thêm N key khác nhau vào một cây BST rỗng để tạo ra một cây có chiều cao N?

3.2.13 (*)Hãy cài lại các hàm `get()` và `put()` cho cây BST mà không dùng đệ quy

Đáp án không đầy đủ : một cài đặt của `get()`:

```
public Value get(Key key)
{
  Node x = root;
  while (x != null)
  {
    int cmp = key.compareTo(x.key);
    if (cmp == 0) return x.val;
    else if (cmp < 0) x = x.left;
    else if (cmp > 0) x = x.right;
  }
  return null;
}
```

Cài đặt của `put()` phức tạp hơn vì cần lưu một con trỏ tới nút cha để có thể nối với nút mới tạo ở tầng đáy. Ngoài ra, vì nhu cầu cập nhật các trường count, bạn còn cần một lần duyệt riêng để kiểm tra xem một key đã có mặt trong cây hay chưa. Do nhu cầu search nhiều hơn nhu cầu insert, đối với các ứng dụng cần tối ưu hóa hiệu năng, cài đặt không đệ quy của hàm `get()` là hữu ích, trong khi đó hàm `put()` không đệ quy không mang lại hiệu ứng đáng kể.

3.2.14 (*)Hãy cài các hàm `min()`, `max()`, `floor()`, `ceiling()`, `rank()`, và `select()`không dùng đệ quy.

3.2.15 Hãy liệt kê chuỗi các nút được duyệt khi các hàm sau của lớp `BST` được chạy trên cây tại hình vẽ bên phải.

a. `floor("Q")`

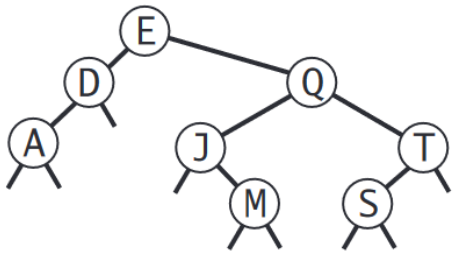
b. `select(5)`

c. `ceiling("Q")`

d. `rank("J")`

e. `size("D", "T")`

f. `keys("D", "T")`



3.2.16 (*) Gọi đường đi ngoài (external path length) của một cây là tổng độ dài của các đường đi từ gốc tới tất cả các link null (độ dài của một đường đi tính theo số nút trên đường). Hãy chứng minh rằng hiệu giữa độ dài đường đi ngoài và đường đi trong (internal path length: tổng độ sâu của các nút) của một cây nhị phân bất kì với N nút là 2N (see Proposition C).

3.2.17 Vẽ các cây BST thu được khi bạn xóa dần từng key ra khỏi cây trong bài 3.2.1, theo thứ tự mà chúng đã được thêm vào cây.

3.2.18 Vẽ các cây BST thu được khi bạn xóa dần từng key ra khỏi cây trong bài 3.2.1, theo thứ tự bảng chữ cái.

3.2.19 Vẽ các cây BST thu được khi bạn xóa dần từng key ra khỏi cây trong bài 3.2.1, mỗi lần xóa key nằm tại nút gốc của cây.

3.2.20 (*) Chứng minh rằng thời gian chạy của hàm `keys()` hai tham số trong một cây BST chứa N nút gần như tỷ lệ thuận với độ cao của cây cộng thêm số key nằm trong khoảng giữa hai tham số.

3.2.21 (*) Bổ sung vào cài đặt cây BST hàm `randomKey()` với nhiệm vụ trả về một key ngẫu nhiên trong bảng tra cứu với thời gian chạy đảm bảo không tồi hơn tỷ lệ thuận với chiều cao của cây.

3.2.22 (*) Chứng minh rằng nếu một nút trong một cây BST có hai con thì nút kế tiếp của nó trong chuỗi tăng dần (successor) không có con trái và nút liền trước nó trong chuỗi tăng dần (predessor) không có con phải.

3.2.23 Phép xóa `delete()` trên cây BST có tính hoán vị không? (nghĩa là “xóa x sau đó xóa y” có cho ra cây giống hệt nếu “xóa y sau đó xóa x”?)

3.2.24 (*) Hãy chứng minh rằng không có thuật toán nào dựa trên các phép so sánh lại có thể xây dựng một cây BST với ít hơn lg(N !) ~ N lg N phép so sánh

3.3.1 Hãy vẽ cây 2-3 thu được khi bạn thêm các key `E A S Y Q U T I O N` theo thứ tự đó vào một cây rỗng.

3.3.2 Hãy vẽ cây 2-3 thu được khi bạn thêm các key `Y L P M X H C R A E S` theo thứ tự đó vào một cây rỗng.

3.3.3 Hãy tìm một thứ tự thêm các key `S E A R C H X M` vào một cây 2-3 rỗng để được một cây 2-3 có độ cao bằng 1.

3.3.4 Hãy chứng minh rằng độ cao của một cây 2-3 với N key nằm trong khoảng từ ~ $\lfloor \log_3 N \rfloor$ 0.63 lg N (với cây toàn nút tam phân) tới ~$\lfloor \log N \rfloor$ (với cây toàn nút nhị



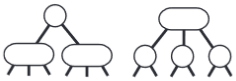
3.3.5 Hình bên phải vẽ tất cả các dạng cây 2-3 với N key, với N từ 1 đến 6 (bỏ qua thứ tự của các cây con). Hãy vẽ tất cả các dạng cây với N = 7, 8, 9, và 10.



3.3.6 (*) Tìm xác suất để mỗi cây 2-3 trong Bài 3.3.5 là kết quả của việc chèn N key ngẫu nhiên phân biệt vào một cây rỗng.

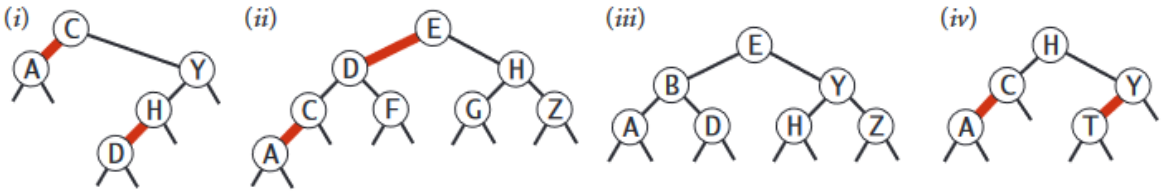


3.3.8 (*) Hãy vẽ tất cả các cách có thể biểu diễn một nút tứ phân bằng ba nút nhị ph nối với nhau bằng các link đỏ (không nhất thiết phải nghiêng trái).



3.3.9 Trong các cây dưới đây, những cây nào là cây đỏ đen?





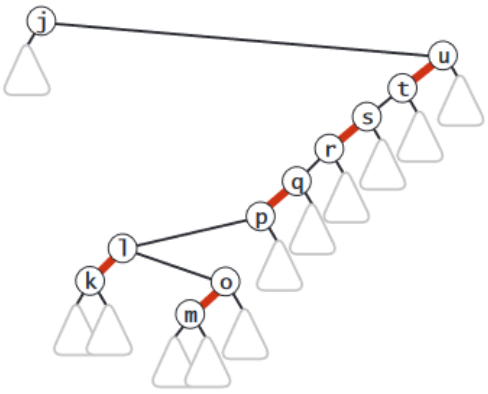
3.3.10 Vẽ cây đỏ đen thu được sau khi chèn thêm các key `E A S Y Q U T I O N` theo thứ tự đó vào một cây rỗng.

3.3.11 Vẽ cây đỏ đen thu được sau khi chèn thêm các key `Y L P M X H C R A E S` theo thứ tự đó vào một cây rỗng.

3.3.13 True or false: Nếu bạn chèn thêm key theo thứ tự tăng dần vào một cây BST đỏ đen, độ cao của cây sẽ không giảm sau mỗi lần thêm key.

3.3.14 Vẽ cây BST đỏ đen thu được sau khi bạn thêm các chữ cái từ `A` đến `K` theo thứ tự đó vào một cây rỗng, sau đó mô tả tổng quát hiện tượng xảy ra khi thêm key vào cây theo thứ tự tăng dần.

3.3.15 Tương tự bài 3.3.14 với các key được chèn thêm theo thứ tự giảm dần.



3.3.16 (*) Hãy vẽ kết quả sau khi chèn thêm `n` vào cây BST đỏ đen bên phải (cây trong hình chỉ vẽ các nút trên đường tìm kiếm n.
Và bạn chỉ phải vẽ các nút đó trong lời giải của mình).

3.3.17 Hãy tạo ra hai chuỗi 16 key tùy ý.
Tạo hai cây BST đỏ đen từ hai chuỗi đó, và hai cây BST
thường (không cân bằng) cũng từ hai chuỗi đó.
So sánh giữa BST đỏ đen và BST thường.

3.3.19 (*)With 1 bit per node for color, we can represent 2-, 3-, and 4-nodes. How many bits per node would we need to represent 5-, 6-, 7-, and 8-nodes with a binary tree?

3.3.20 (*)Tính internal path length (tổng độ sâu của các nút) của một cây BST cân bằng hoàn hảo gồm N nút với N là một lũy thừa của 2 trừ đi 1.

3.3.22 Tìm một chuỗi key sao cho khi chèn chuỗi đó vào một cây BST thường và một cây BST đỏ đen ta có độ cao của cây BST thường nhỏ hơn độ cao của cây BST đỏ đen. Hoặc chứng minh rằng không tồn tại một chuỗi như vậy.